CodInClub

Powered By BridgeLabz

Best Programming Practices

Encapsulation

- Use `private` access modifiers for class fields to restrict direct access.
- Provide `public` getter and setter methods to access and modify private fields.
- Implement validation logic in setters to ensure data integrity.
- Use `final` fields and avoid setters for immutable classes.
- Follow naming conventions for methods (e.g., `getX`, `setX`).

Polymorphism

- Program to an interface, not an implementation.
- Ensure overridden methods adhere to the base class method's contract.
- Avoid explicit casting; rely on polymorphic behavior.
- Leverage covariant return types for overriding methods.
- Keep inheritance hierarchies shallow to maintain simplicity.

Interfaces

- Use interfaces to define a contract or behavior.
- Prefer default methods only when backward compatibility or shared implementation is necessary.
- Combine interfaces to create modular, reusable behaviors.
- Favor composition over inheritance when combining multiple behaviors.

Abstract Classes

- Use abstract classes for shared state and functionality among related classes.
- Avoid overusing abstract classes; use them only when clear shared behavior exists.
- Combine abstract classes with interfaces to separate behavior and implementation.
- Avoid deep inheritance hierarchies; keep designs flexible and maintainable.

General Practices

- Follow Java naming conventions for classes, methods, and variables.
- Document code with comments and Javadoc to improve readability.
- Ensure consistency and readability by adhering to team or industry coding standards.
- Apply SOLID principles, particularly Single Responsibility and Interface Segregation. (We will learn it in coming days)

Tips for Implementation

- **Encapsulation**: Ensure all sensitive fields are private and accessed through well-defined getter and setter methods. Include validation logic where applicable.
- **Polymorphism**: Use abstract class references or interface references to handle objects of multiple types dynamically.
- **Abstract Classes**: Use them to define a common structure and behavior while deferring specific details to subclasses.
- **Interfaces**: Use them to define additional capabilities or contracts that are not tied to the class hierarchy.

Problem Statements

1. Employee Management System

- **Description**: Build an employee management system with the following requirements:
  - Use an abstract class `Employee` with fields like `employeeId`, `name`, and `baseSalary`.
  - Provide an abstract method `calculateSalary()` and a concrete method `displayDetails()`.
  - Create two subclasses: `FullTimeEmployee` and `PartTimeEmployee`, implementing `calculateSalary()` based on work hours or fixed salary.
  - Use encapsulation to restrict direct access to fields and provide getter and setter methods.
  - Create an interface `Department` with methods like `assignDepartment()` and `getDepartmentDetails()`.
  - Ensure polymorphism by processing a list of employees and displaying their details using the `Employee` reference.

```
package com.employeemanagementsystem;

abstract class Employee {
   //fields
   private int employeeId;
   private String name;
   private double baseSalary;

   //abstract method to calculate salary
   public abstract double calculateSalary();

   //constructor to initialize values
```

```
    public Employee(int employeeId, String name, double baseSalary) {
        this.employeeId = employeeId;
        this.name = name;
        this. baseSalary = baseSalary;
    }

    //method to display details
    public void display() {
        System.out.println("Employee Id : " + this.employeeId);
        System.out.println("Name : " + this.name);
        System.out.println("Base salary : " + this.baseSalary);
    }
    //setter methods
    public void setEmployeeId(int employeeId) {
        this.employeeId = employeeId;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setBaseSalary(double baseSalary) {
        this.baseSalary = baseSalary;
    }

    //getter methods
    public int getEmployeeId() {
        return this.employeeId;
    }
    public String getName() {
        return this.name;
    }
    public double getBaseSalary() {
        return baseSalary;
    }
}
```

```
package com.employeemanagementsystem;

public interface Department {
    //method to assign department to be implemented in derived classes
    void assignDepartment(String departmentName);
```

```
    //method to get department details to be implemented in derived classes
    String getDepartmentDetails();
}
```

```
package com.employeemanagementsystem;

public class FullTimeEmployee extends Employee implements Department {
    //fields
    private double fixedSalary;
    private String departmentName;

    //constructor to initialize value
    public FullTimeEmployee(int employeeId, String name, double baseSalary, double
fixedSalary) {
        super(employeeId, name, baseSalary);
        this.fixedSalary = fixedSalary;
    }
    //implementing calculateSalary() method
    @Override
    public double calculateSalary() {
        return this.getBaseSalary() + this.fixedSalary;
    }
    //implementing assignDepartment() method
    @Override
    public void assignDepartment(String departmentName) {
        this.departmentName = departmentName;
    }
    //implementing getDepartmentDetails() method
    public String getDepartmentDetails() {
        return this.departmentName;
    }

    //setter methods
    public void setFixedSalary(double fixedSalary) {
        this.fixedSalary = fixedSalary;
    }
    //getter methods
    public double getFixedSalary() {
        return this.fixedSalary;
    }
}
```

```java
package com.employeemanagementsystem;

public class PartTimeEmployee extends Employee implements Department {
   //fields
   private double hourlyRate;
   private double workHours;
   private String departmentName;

   //constructor to initialize value
   public PartTimeEmployee(int employeeId, String name, double baseSalary, double
hourlyRate, double workHours) {
       super(employeeId, name, baseSalary);
       this.hourlyRate = hourlyRate;
       this.workHours = workHours;
   }
   //implementing calculateSalary() method
   @Override
   public double calculateSalary() {
       return this.getBaseSalary() + (this.workHours * this.hourlyRate);
   }
   //implementing assignDepartment() method
   @Override
   public void assignDepartment(String departmentName) {
       this.departmentName = departmentName;
   }
   //implementing getDepartmentDetails() method
   public String getDepartmentDetails() {
       return this.departmentName;
   }

   //setter methods
   public void setWorkHours(double workHours) {
       this.workHours = workHours;
   }
   public void setHourlyRate(double hourlyRate) {
       this.hourlyRate = hourlyRate;
   }

   //getter methods
   public double getWorkHours() {
       return this.workHours;
   }
   public double getHourlyRate() {
       return this.hourlyRate;
   }
```

```
}
```

```
package com.employeemanagementsystem;
import java.util.ArrayList;
import java.util.List;

public class Controller {
   public static void main(String[] args) {
       //Creating a list of employees
       List<Employee> employees = new ArrayList<>();

       // Adding FullTimeEmployee
       FullTimeEmployee fullTimeEmployee = new FullTimeEmployee(1, "Deepanshu
malviya", 30000, 20000);
       fullTimeEmployee.assignDepartment("Finance");
       employees.add(fullTimeEmployee);

       // Adding PartTimeEmployee
       PartTimeEmployee partTimeEmployee = new PartTimeEmployee(2, "Shubham kumar",
15000, 80, 200);
       partTimeEmployee.assignDepartment("HR");
       employees.add(partTimeEmployee);

       // Processing the list of employees
       for (Employee employee : employees) {
           employee.display();
           if (employee instanceof Department) {
               System.out.println(((Department) employee).getDepartmentDetails());
           }
           System.out.println("-----------------------------------");
       }
   }
}
```

---

2. E-Commerce Platform

- **Description**: Develop a simplified e-commerce platform:

- Create an abstract class `Product` with fields like `productId`, `name`, and `price`, and an abstract method `calculateDiscount()`.
- Extend it into concrete classes: `Electronics`, `Clothing`, and `Groceries`.
- Implement an interface `Taxable` with methods `calculateTax()` and `getTaxDetails()` for applicable product categories.
- Use encapsulation to protect product details, allowing updates only through setter methods.
- Showcase polymorphism by creating a method that calculates and prints the final price (price + tax - discount) for a list of `Product`.

```
package com.ecommerceplatform;

abstract class Product {
   //fields
   private int productId;
   private String productName;
   private double price;

   //constructor to initialize values
   public Product(int productId, String productName, double price) {
       this.productId = productId;
       this.productName = productName;
       this.price = price;
   }
   //abstract method to calculate discount
   public abstract double calculateDiscount();

   //method to display details
   public void display() {
       System.out.println("Product ID : " + this.productId);
       System.out.println("Product name : " + this.productName);
   }
   //setter methods
   public void setProductId(int productId) {
       this.productId = productId;
   }
   public void setProductName(String productName) {
       this.productName = productName;
   }

   public void setPrice(double price) {
       this.price = price;
   }
```

```
    //getter methods
    public int getProductId() {
        return productId;
    }
    public String getProductName() {
        return productName;
    }

    public double getPrice() {
        return price;
    }
}
```

```
package com.ecommerceplatform;

interface Taxable {

    //method to calculate tax to be implemented in drived class
    double calculateTax();

    //method to get tax details to be implemented in drived class
    String getTaxDetails();
}
```

```
package com.ecommerceplatform;

class Clothing extends Product implements Taxable {

    //fields
    private static final double CLOTHING_TAX_RATE = 0.05; // 5% tax
    private static final double CLOTHING_DISCOUNT_RATE = 0.15; // 15% discount

    //constructor to initialize values
    public Clothing(int productId, String name, double price) {
        super(productId, name, price);
    }

    //implementation of calculateDiscount() method
    @Override
    public double calculateDiscount() {
```

```
        return getPrice() * CLOTHING_DISCOUNT_RATE;
    }

    @Override
    public double calculateTax() {
        return getPrice() * CLOTHING_TAX_RATE;
    }

    @Override
    public String getTaxDetails() {
        return "Clothing Tax Rate: 5%";
    }
}
```

```
package com.ecommerceplatform;

// Subclass: Electronics
class Electronics extends Product implements Taxable {
    //fields
    private static final double ELECTRONICS_TAX_RATE = 0.18; // 18% tax
    private static final double ELECTRONICS_DISCOUNT_RATE = 0.10; // 10% discount

    //constructor to initialize values
    public Electronics(int productId, String name, double price) {
        super(productId, name, price);
    }
    //implementation of calculateDiscount() method
    @Override
    public double calculateDiscount() {
        return getPrice() * ELECTRONICS_DISCOUNT_RATE;
    }
    //implementation of calculateTax() method
    @Override
    public double calculateTax() {
        return getPrice() * ELECTRONICS_TAX_RATE;
    }
    //implementation of getTaxDetails() method
    @Override
    public String getTaxDetails() {
        return "Electronics Tax Rate: 18%";
    }
}
```

```java
package com.ecommerceplatform;

class Groceries extends Product {
   private static final double GROCERIES_DISCOUNT_RATE = 0.05; // 5% discount

   //constructor to initialize values
   public Groceries(int productId, String name, double price) {
       super(productId, name, price);
   }

   //implementation of calculateDiscount() method
   @Override
   public double calculateDiscount() {
       return getPrice() * GROCERIES_DISCOUNT_RATE;
   }
}
```

```java
package com.ecommerceplatform;
import java.util.ArrayList;
import java.util.List;

public class Controller {
   public static void main(String[] args) {

       // Creating a list of products
       List<Product> products = new ArrayList<>();

       products.add(new Electronics(101, "Laptop", 50000));
       products.add(new Clothing(201, "T-Shirt", 1000));
       products.add(new Groceries(301, "Apples", 200));

       // Calculating and printing the final price for each product
       for (Product product : products) {
           double price = product.getPrice();
           double discount = product.calculateDiscount();
           double tax = product instanceof Taxable ? ((Taxable)
product).calculateTax() : 0.0;
           double finalPrice = price + tax - discount;

           System.out.println("Product Name: " + product.getProductName());
           System.out.println("Base Price: " + price);
           System.out.println("Discount: -" + discount);
           System.out.println("Tax: +" + tax);
```

```
            System.out.println("Final Price: " + finalPrice);

            // Displaying tax details for taxable products
            if (product instanceof Taxable) {
                System.out.println(((Taxable) product).getTaxDetails());
            }
            System.out.println("-----------------------------------");
        }
    }
}
```

---

3. Vehicle Rental System

- **Description**: Design a system to manage vehicle rentals:
    - Define an abstract class `Vehicle` with fields like `vehicleNumber`, `type`, and `rentalRate`.
    - Add an abstract method `calculateRentalCost(int days)`.
    - Create subclasses `Car`, `Bike`, and `Truck` with specific implementations of `calculateRentalCost()`.
    - Use an interface `Insurable` with methods `calculateInsurance()` and `getInsuranceDetails()`.
    - Apply encapsulation to restrict access to sensitive details like insurance policy numbers.
    - Demonstrate polymorphism by iterating over a list of vehicles and calculating rental and insurance costs for each.

```
package com.vehiclerentalsystem;

abstract class Vehicle {
    private String vehicleNumber;
    private String type;
    private double rentalRate;

    // Constructor to initialize values
    public Vehicle(String vehicleNumber, String type, double rentalRate) {
        this.vehicleNumber = vehicleNumber;
        this.type = type;
        this.rentalRate = rentalRate;
```

```
    }

    // Abstract Method to calculate rental cost to be implemented in derived class
    public abstract double calculateRentalCost(int days);

    // Getter methods
    public String getVehicleNumber() {
        return vehicleNumber;
    }
    public String getType() {
        return type;
    }
    public double getRentalRate() {
        return rentalRate;
    }
}
```

```
package com.vehiclerentalsystem;

public interface Insurable {

    //abstract method to calculate insurance to be implemented in derived class
    double calculateInsurance();

    //abstract method to get insurance details to be implemented in derived class
    String getInsuranceDetails();
}
```

```
package com.vehiclerentalsystem;

class Car extends Vehicle implements Insurable {
    //fields
    private static final double INSURANCE_RATE = 500; // Fixed insurance cost for
cars

    //constructor to initialize values
    public Car(String vehicleNumber, double rentalRate) {
        super(vehicleNumber, "Car", rentalRate);
    }
```

```
    //implementation of calculateRentalCost() method
    @Override
    public double calculateRentalCost(int days) {
        return getRentalRate() * days;
    }

    //implementation of calculateInsurance() method
    @Override
    public double calculateInsurance() {
        return INSURANCE_RATE;
    }

    //implementation of getInsuranceDetails() method
    @Override
    public String getInsuranceDetails() {
        return "Car Insurance: Fixed cost of " + INSURANCE_RATE;
    }
}
```

```
package com.vehiclerentalsystem;


class Bike extends Vehicle implements Insurable {
    private static final double INSURANCE_RATE = 200; // Fixed insurance cost for
bikes

    //constructor to initialize values
    public Bike(String vehicleNumber, double rentalRate) {
        super(vehicleNumber, "Bike", rentalRate);
    }

    //implementation of calculateRentalCost() method
    @Override
    public double calculateRentalCost(int days) {
        return getRentalRate() * days;
    }

    //implementation of calculateInsurance() method
    @Override
    public double calculateInsurance() {
        return INSURANCE_RATE;
    }
```

```
    //implementation of getInsuranceDetails() method
    @Override
    public String getInsuranceDetails() {
        return "Bike Insurance: Fixed cost of " + INSURANCE_RATE;
    }
}
```

```
package com.vehiclerentalsystem;

// Subclass: Truck
class Truck extends Vehicle implements Insurable {
    private static final double INSURANCE_RATE = 1000; // Fixed insurance cost for
trucks

    //constructor to initialize values
    public Truck(String vehicleNumber, double rentalRate) {
        super(vehicleNumber, "Truck", rentalRate);
    }

    //implementation of calculateRentalCost() method
    @Override
    public double calculateRentalCost(int days) {
        return getRentalRate() * days;
    }

    //implementation of calculateInsurance() method
    @Override
    public double calculateInsurance() {
        return INSURANCE_RATE;
    }

    //implementation of getInsuranceDetails() method
    @Override
    public String getInsuranceDetails() {
        return "Truck Insurance: Fixed cost of " + INSURANCE_RATE;
    }
}
```

```
package com.vehiclerentalsystem;
```

```
import java.util.ArrayList;
import java.util.List;

public class Controller {
   public static void main(String[] args) {
       // Creating a list of vehicles (Polymorphism in action)
       List<Vehicle> vehicles = new ArrayList<>();
       vehicles.add(new Car("CAR123", 1000));
       vehicles.add(new Bike("BIKE333", 500));
       vehicles.add(new Truck("TRUCK333", 2000));

       // Iterating over the vehicles and calculating rental and insurance costs
       int rentalDays = 3; //for 3 days
       for (Vehicle vehicle : vehicles) {
           double rentalCost = vehicle.calculateRentalCost(rentalDays);
           double insuranceCost = ((Insurable) vehicle).calculateInsurance();

           System.out.println("Vehicle Type: " + vehicle.getType());
           System.out.println("Vehicle Number: " + vehicle.getVehicleNumber());
           System.out.println("Rental Cost for " + rentalDays + " days: " +
rentalCost);
           System.out.println("Insurance Cost: " + insuranceCost);
           System.out.println(((Insurable) vehicle).getInsuranceDetails());
           System.out.println("---------------------------------------");
       }
   }
}
```

---

4. Banking System

- **Description**: Create a banking system with different account types:
  - Define an abstract class `BankAccount` with fields like `accountNumber`, `holderName`, and `balance`.
  - Add methods like `deposit(double amount)` and `withdraw(double amount)` (concrete) and `calculateInterest()` (abstract).
  - Implement subclasses `SavingsAccount` and `CurrentAccount` with unique interest calculations.
  - Create an interface `Loanable` with methods `applyForLoan()` and `calculateLoanEligibility()`.
  - Use encapsulation to secure account details and restrict unauthorized access.

- Demonstrate polymorphism by processing different account types and calculating interest dynamically.

```
package com.bankingsystem;

abstract class BankAccount {
   //fields
   private String accountNumber;
   private String holderName;
   private double balance;

   // Constructor to initialize values
   public BankAccount(String accountNumber, String holderName, double balance) {
       this.accountNumber = accountNumber;
       this.holderName = holderName;
       this.balance = balance;
   }
   // Concrete Method: Deposit
   public void deposit(double amount) {
       if (amount > 0) {
           balance += amount;
           System.out.println(amount + " deposited successfully. Current Balance: "
+ balance);
       } else {
           System.out.println("Invalid deposit amount.");
       }
   }

   // Concrete Method: method to withdraw money
   public void withdraw(double amount) {
       if (amount > 0 && amount <= balance) {
           balance -= amount;
           System.out.println(amount + " withdrawn successfully. Current Balance: "
+ balance);
       } else {
           System.out.println("Invalid withdrawal amount or insufficient
balance.");
       }
   }

   // Abstract Method: to calculate interest
   public abstract double calculateInterest();

   // Getters
   public String getAccountNumber() {
```

```
        return accountNumber;
    }

    public String getHolderName() {
        return holderName;
    }

    public double getBalance() {
        return balance;
    }

    // Setters
    public void setBalance(double balance) {
        this.balance = balance;
    }
}
```

```
package com.bankingsystem;

interface Loanable {

    //method to apply for loan to be implemented in derived class
    void applyForLoan(double amount);

    //method to calculate loan eligibility to be implemented in derived class
    boolean calculateLoanEligibility();
}
```

```
package com.bankingsystem;

class CurrentAccount extends BankAccount implements Loanable {
    private static final double INTEREST_RATE = 0.02; // 2% annual interest
    private boolean loanEligible;

    // Constructor to initialize values
    public CurrentAccount(String accountNumber, String holderName, double balance) {
        super(accountNumber, holderName, balance);
        loanEligible = balance > 10000; // Eligibility based on balance
    }
```

```
    //implementation of calculateInterest() method
    @Override
    public double calculateInterest() {
        return getBalance() * INTEREST_RATE;
    }

    //implementation of applyForLoan() method
    @Override
    public void applyForLoan(double amount) {
        if (loanEligible) {
            System.out.println("Loan of " + amount + " approved for Current Account
holder: " + getHolderName());
        } else {
            System.out.println("Loan application denied. Insufficient balance for
eligibility.");
        }
    }

    //implementation of calculateLoanEligibility() method
    @Override
    public boolean calculateLoanEligibility() {
        return loanEligible;
    }
}
```

```
package com.bankingsystem;

class SavingsAccount extends BankAccount implements Loanable {
    private static final double INTEREST_RATE = 0.04; // 4% annual interest
    private boolean loanEligible;

    // Constructor to initialize values
    public SavingsAccount(String accountNumber, String holderName, double balance) {
        super(accountNumber, holderName, balance);
        loanEligible = balance > 5000; // Eligibility based on balance
    }

    //implementation of calculateInterest() method
    @Override
    public double calculateInterest() {
        return getBalance() * INTEREST_RATE;
    }
```

```
    //implementation of applyForLoan() method
    @Override
    public void applyForLoan(double amount) {
        if (loanEligible) {
            System.out.println("Loan of " + amount + " approved for Savings Account
holder: " + getHolderName());
        } else {
            System.out.println("Loan application denied. Insufficient balance for
eligibility.");
        }
    }

    //implementation of calculateLoanEligibility() method
    @Override
    public boolean calculateLoanEligibility() {
        return loanEligible;
    }
}
```

```
package com.bankingsystem;

public class Controller {
    public static void main(String[] args) {
        // Creating accounts
        BankAccount savingsAccount = new SavingsAccount("SA12345", "Deepanshu",
8000);
        BankAccount currentAccount = new CurrentAccount("CA54321", "Karan", 15000);

        // Processing saving account
        System.out.println("Processing Savings Account:");
        savingsAccount.deposit(2000);
        savingsAccount.withdraw(1000);
        System.out.println("Interest Earned: " +
savingsAccount.calculateInterest());
        ((Loanable) savingsAccount).applyForLoan(5000);
        System.out.println("-----------------------------------");

        // Processing saving account
        System.out.println("Processing Current Account:");
        currentAccount.deposit(5000);
        currentAccount.withdraw(3000);
        System.out.println("Interest Earned: " +
currentAccount.calculateInterest());
```

```
        ((Loanable) currentAccount).applyForLoan(10000);
        System.out.println("-------------------------------------");
    }
}
```

---

5. Library Management System

- **Description**: Develop a library management system:
    - Use an abstract class `LibraryItem` with fields like `itemId`, `title`, and `author`.
    - Add an abstract method `getLoanDuration()` and a concrete method `getItemDetails()`.
    - Create subclasses `Book`, `Magazine`, and `DVD`, overriding `getLoanDuration()` with specific logic.
    - Implement an interface `Reservable` with methods `reserveItem()` and `checkAvailability()`.
    - Apply encapsulation to secure details like the borrower's personal data.
    - Use polymorphism to allow a general `LibraryItem` reference to manage all items, regardless of type.

```
package com.librarymanagementsystem;

abstract class LibraryItem {
    private String itemId;
    private String title;
    private String author;

    // Constructor to initialize values
    public LibraryItem(String itemId, String title, String author) {
        this.itemId = itemId;
        this.title = title;
        this.author = author;
    }

    // Concrete Method: to Get Item Details
    public String getItemDetails() {
        return "Item ID: " + itemId + ", Title: " + title + ", Author: " + author;
```

```
    }

    // Abstract Method: to Get Loan Duration
    public abstract int getLoanDuration();

    // Getters
    public String getItemId() {
        return itemId;
    }

    public String getTitle() {
        return title;
    }

    public String getAuthor() {
        return author;
    }
}
```

```
package com.librarymanagementsystem;

interface Reservable {

    //method to reverse items
    void reserveItem(String borrowerName);
    //method to check availability
    boolean checkAvailability();
}
```

```
package com.librarymanagementsystem;

class Book extends LibraryItem implements Reservable {
    private boolean isAvailable = true;

    // Constructor to initialize values
    public Book(String itemId, String title, String author) {
        super(itemId, title, author);
    }

    //implementation of getLoanDuration() method
```

```
    @Override
    public int getLoanDuration() {
        return 14; // Books can be borrowed for 14 days
    }

    //implementation of reverseItem() method
    @Override
    public void reserveItem(String borrowerName) {
        if (isAvailable) {
            isAvailable = false;
            System.out.println("Book reserved for " + borrowerName);
        } else {
            System.out.println("Book is currently not available.");
        }
    }

    //implementation of checkAvailability() method
    @Override
    public boolean checkAvailability() {
        return isAvailable;
    }
}
```

```
package com.librarymanagementsystem;

class Magazine extends LibraryItem implements Reservable {
    private boolean isAvailable = true;

    // Constructor to initialize values
    public Magazine(String itemId, String title, String author) {
        super(itemId, title, author);
    }


    //implementation of getLoanDuration() method
    @Override
    public int getLoanDuration() {
        return 7; // Magazines can be borrowed for 7 days
    }

    //implementation of reverseItem() method
    @Override
    public void reserveItem(String borrowerName) {
```

```
        if (isAvailable) {
            isAvailable = false;
            System.out.println("Magazine reserved for " + borrowerName);
        } else {
            System.out.println("Magazine is currently not available.");
        }
    }

    //implementation of checkAvailability() method
    @Override
    public boolean checkAvailability() {
        return isAvailable;
    }
}
```

```
package com.librarymanagementsystem;

class DVD extends LibraryItem implements Reservable {
    private boolean isAvailable = true;

    // Constructor to initialize values
    public DVD(String itemId, String title, String author) {
        super(itemId, title, author);
    }

    //implementation of getLoanDuration() method
    @Override
    public int getLoanDuration() {
        return 5; // DVDs can be borrowed for 5 days
    }

    //implementation of reverseItem() method
    @Override
    public void reserveItem(String borrowerName) {
        if (isAvailable) {
            isAvailable = false;
            System.out.println("DVD reserved for " + borrowerName);
        } else {
            System.out.println("DVD is currently not available.");
        }
    }

    //implementation of checkAvailability() method
```

```
    @Override
    public boolean checkAvailability() {
        return isAvailable;
    }
}
```

```
package com.librarymanagementsystem;

public class Controller {
    public static void main(String[] args) {
        // Creating Library Items
        LibraryItem book = new Book("B001", "Effective Java", "Joshua Bloch");
        LibraryItem magazine = new Magazine("M001", "National Geographic",
"Editorial Team");
        LibraryItem dvd = new DVD("D001", "Inception", "Christopher Nolan");

        // Processing Library Items
        LibraryItem[] libraryItems = {book, magazine, dvd};
        for (LibraryItem item : libraryItems) {
            System.out.println(item.getItemDetails());
            System.out.println("Loan Duration: " + item.getLoanDuration() + "
days");
            System.out.println("----------------------------------------");
        }

        // Reserving Items
        Reservable reservableBook = (Reservable) book;
        Reservable reservableMagazine = (Reservable) magazine;
        Reservable reservableDVD = (Reservable) dvd;

        reservableBook.reserveItem("Shubham");
        reservableMagazine.reserveItem("Raj");
        reservableDVD.reserveItem("Gagan");

        // Checking Availability
        System.out.println("\nChecking availability after reservation:");
        System.out.println("Book available: " + reservableBook.checkAvailability());
        System.out.println("Magazine available: " +
reservableMagazine.checkAvailability());
        System.out.println("DVD available: " + reservableDVD.checkAvailability());
    }
}
```

---

6. Online Food Delivery System

- **Description**: Create an online food delivery system:
  - Define an abstract class `FoodItem` with fields like `itemName`, `price`, and `quantity`.
  - Add abstract methods `calculateTotalPrice()` and concrete methods like `getItemDetails()`.
  - Extend it into classes `VegItem` and `NonVegItem`, overriding `calculateTotalPrice()` to include additional charges (e.g., for non-veg items).
  - Use an interface `Discountable` with methods `applyDiscount()` and `getDiscountDetails()`.
  - Demonstrate encapsulation to restrict modifications to order details and use polymorphism to handle different types of food items in a single order-processing method.

```
package com.onlinefooddeliverysystem;

abstract class FoodItem {
   //fields
   private String itemName;
   private double price;
   private int quantity;

   // Constructor to initialize values
   public FoodItem(String itemName, double price, int quantity) {
       this.itemName = itemName;
       this.price = price;
       this.quantity = quantity;
   }

   // Abstract Method to Calculate Total Price
   public abstract double calculateTotalPrice();

   // Method to Get Item Details
   public String getItemDetails() {
       return "Item Name: " + itemName + ", Price: " + price + ", Quantity: " +
quantity;
   }
```

```
    // Getters
    public String getItemName() {
        return itemName;
    }

    public double getPrice() {
        return price;
    }

    public int getQuantity() {
        return quantity;
    }

    // Setters
    public void setQuantity(int quantity) {
        if (quantity > 0) {
            this.quantity = quantity;
        } else {
            System.out.println("Quantity must be positive!");
        }
    }
}
```

```
package com.onlinefooddelivery system;

interface Discountable {
    //method to apply discount to be implemented in derived class
    void applyDiscount(double discountPercentage);
    //method to get discount details to be implemented in derived class
    String getDiscountDetails();
}
```

```
package com.onlinefooddelivery system;

class VegItem extends FoodItem implements Discountable {
    private double discount = 0.0;

    // Constructor to initialize values
    public VegItem(String itemName, double price, int quantity) {
```

```
        super(itemName, price, quantity);
    }
    //implementation of calculateTotalPrice() method
    @Override
    public double calculateTotalPrice() {
        return (getPrice() * getQuantity()) - discount;
    }
    //implementation of applyDiscount() method
    @Override
    public void applyDiscount(double discountPercentage) {
        this.discount = (getPrice() * getQuantity() * discountPercentage) / 100;
    }
    //implementation of getDiscountDetails() method
    @Override
    public String getDiscountDetails() {
        return "Discount on Veg Item: $" + discount;
    }
}
```

```
package com.onlinefooddeliverysystem;

class NonVegItem extends FoodItem implements Discountable {
    private double discount = 0.0;
    private static final double NON_VEG_EXTRA_CHARGE = 10.0;

    // Constructor to initialize values
    public NonVegItem(String itemName, double price, int quantity) {
        super(itemName, price, quantity);
    }
    //implementation of calculateTotalPrice() method
    @Override
    public double calculateTotalPrice() {
        return (getPrice() * getQuantity() + NON_VEG_EXTRA_CHARGE) - discount;
    }
    //implementation of applyDiscount() method
    @Override
    public void applyDiscount(double discountPercentage) {
        this.discount = (getPrice() * getQuantity() * discountPercentage) / 100;
    }
    //implementation of getDiscountDetails() method
    @Override
    public String getDiscountDetails() {
        return "Discount on Non-Veg Item: $" + discount;
```

```
    }
}
```

```
package com.onlinefooddeliverysystem;

public class Controller {
    public static void main(String[] args) {
        // Creating Food Items
        FoodItem vegItem = new VegItem("Paneer Butter Masala", 200.0, 2);
        FoodItem nonVegItem = new NonVegItem("Chicken Biryani", 300.0, 1);

        // Applying Discounts
        ((Discountable) vegItem).applyDiscount(10); // 10% discount on veg item
        ((Discountable) nonVegItem).applyDiscount(5); // 5% discount on non-veg item

        // Processing Order
        FoodItem[] order = {vegItem, nonVegItem};
        double totalOrderPrice = 0;

        //printing details
        for (FoodItem item : order) {
            System.out.println(item.getItemDetails());
            System.out.println(((Discountable) item).getDiscountDetails());
            System.out.println("Total Price: $" + item.calculateTotalPrice());
            totalOrderPrice += item.calculateTotalPrice();
            System.out.println("----------------------------------------");
        }
        //printing the total order price
        System.out.println("Total Order Price: $" + totalOrderPrice);
    }
}
```

---

7. Hospital Patient Management

- **Description**: Design a system to manage patients in a hospital:
    - Create an abstract class `Patient` with fields like `patientId`, `name`, and `age`.
    - Add an abstract method `calculateBill()` and a concrete method `getPatientDetails()`.

- Extend it into subclasses `InPatient` and `OutPatient`, implementing `calculateBill()` with different billing logic.
- Implement an interface `MedicalRecord` with methods `addRecord()` and `viewRecords()`.
- Use encapsulation to protect sensitive patient data like diagnosis and medical history.
- Use polymorphism to handle different patient types and display their billing details dynamically.

```
package com.hospitalpatientmanagement;

abstract class Patient {
   private int patientId;
   private String name;
   private int age;

   //constructor to initialize values
   public Patient(int patientId, String name, int age) {
       this.patientId = patientId;
       this.name = name;
       this.age = age;
   }

   // Abstract Method to Calculate Bill
   public abstract double calculateBill();

   //Method to Get Patient Details
   public String getPatientDetails() {
       return "Patient ID: " + patientId + ", Name: " + name + ", Age: " + age;
   }

   // Getters
   public int getPatientId() {
       return patientId;
   }

   public String getName() {
       return name;
   }

   public int getAge() {
       return age;
   }
}
```

```java
package com.hospitalpatientmanagement;

interface MedicalRecord {
   //method to add Record to be implemented in derived class
   void addRecord(String record);

   //method to view Record to be implemented in derived class
   String viewRecords();
}
```

```java
package com.hospitalpatientmanagement;

class InPatient extends Patient implements MedicalRecord {
   private double roomCharges;
   private double treatmentCost;
   private String medicalHistory;

   //constructor to initialize values
   public InPatient(int patientId, String name, int age, double roomCharges, double
treatmentCost) {
       super(patientId, name, age);
       this.roomCharges = roomCharges;
       this.treatmentCost = treatmentCost;
   }

   //implementation of calculateBill() method
   @Override
   public double calculateBill() {
       return roomCharges + treatmentCost;
   }

   //implementation of addRecord() method
   @Override
   public void addRecord(String record) {
       medicalHistory = (medicalHistory == null) ? record : medicalHistory + "; " +
record;
   }

   //implementation of viewRecord() method
   @Override
   public String viewRecords() {
       return medicalHistory == null ? "No medical history available." :
```

```
medicalHistory;
    }
}
```

```
package com.hospitalpatientmanagement;

class OutPatient extends Patient implements MedicalRecord {
    private double consultationFee;
    private double medicationCost;
    private String diagnosis;

    //constructor to initialize values
    public OutPatient(int patientId, String name, int age, double consultationFee,
double medicationCost) {
        super(patientId, name, age);
        this.consultationFee = consultationFee;
        this.medicationCost = medicationCost;
    }

    //implementation of calculateBill() method
    @Override
    public double calculateBill() {
        return consultationFee + medicationCost;
    }

    //implementation of addRecord() method
    @Override
    public void addRecord(String record) {
        diagnosis = (diagnosis == null) ? record : diagnosis + "; " + record;
    }

    //implementation of viewRecord() method
    @Override
    public String viewRecords() {
        return diagnosis == null ? "No diagnosis available." : diagnosis;
    }
}
```

```
package com.hospitalpatientmanagement;
```

```
public class Controller {
   public static void main(String[] args) {
       // Creaing Patients
       Patient inPatient = new InPatient(101, "Raj", 45, 5000, 15000);
       Patient outPatient = new OutPatient(102, "Shubham", 30, 500, 200);

       // Adding Medical Records
       ((MedicalRecord) inPatient).addRecord("Admitted for surgery, treated
successfully.");
       ((MedicalRecord) outPatient).addRecord("Consulted for flu, prescribed
medication.");

       // Processing Patients
       Patient[] patients = {inPatient, outPatient};
       for (Patient patient : patients) {
           System.out.println(patient.getPatientDetails());
           System.out.println("Medical History: " + ((MedicalRecord)
patient).viewRecords());
           System.out.println("Total Bill: $" + patient.calculateBill());
           System.out.println("----------------------------------------");
       }
   }
}
```

---

8. Ride-Hailing Application

- **Description**: Develop a ride-hailing application:
  - Define an abstract class `Vehicle` with fields like `vehicleId`, `driverName`, and `ratePerKm`.
  - Add abstract methods `calculateFare(double distance)` and a concrete method `getVehicleDetails()`.
  - Create subclasses `Car`, `Bike`, and `Auto`, overriding `calculateFare()` based on type-specific rates.
  - Use an interface `GPS` with methods `getCurrentLocation()` and `updateLocation()`.
  - Secure driver and vehicle details using encapsulation.
  - Demonstrate polymorphism by creating a method to calculate fares for different vehicle types dynamically.

```
package com.ridehailingapplication;

abstract class Vehicle {
   private String vehicleId;
   private String driverName;
   private double ratePerKm;

   // Constructor to initialize values
   public Vehicle(String vehicleId, String driverName, double ratePerKm) {
       this.vehicleId = vehicleId;
       this.driverName = driverName;
       this.ratePerKm = ratePerKm;
   }

   // Abstract Method to Calculate Fare
   public abstract double calculateFare(double distance);

   // Method to Get Vehicle Details
   public String getVehicleDetails() {
       return "Vehicle ID: " + vehicleId + ", Driver: " + driverName + ", Rate per
Km: $" + ratePerKm;
   }

   // Getters
   public String getVehicleId() {
       return vehicleId;
   }

   public String getDriverName() {
       return driverName;
   }

   public double getRatePerKm() {
       return ratePerKm;
   }
}
```

```
package com.ridehailingapplication;

interface GPS {

   //method to get current location to be implemented in derived class
   String getCurrentLocation();
```

```
    //method to update location to be implemented in derived class
    void updateLocation(String location);
}
```

```
package com.ridehailingapplication;

class Car extends Vehicle implements GPS {
    private String currentLocation;

    // Constructor to initialize values
    public Car(String vehicleId, String driverName, double ratePerKm) {
        super(vehicleId, driverName, ratePerKm);
    }

    //implementation of calculateFare() method
    @Override
    public double calculateFare(double distance) {
        return getRatePerKm() * distance + 10; // Adding a base charge for car rides
    }

    //implementation of getCurrentLocation() method
    @Override
    public String getCurrentLocation() {
        return currentLocation;
    }

    //implementation of updateLocaation() method
    @Override
    public void updateLocation(String location) {
        this.currentLocation = location;
    }
}
```

```
package com.ridehailingapplication;

class Bike extends Vehicle implements GPS {
    private String currentLocation;
```

```
    // Constructor to initialize values
    public Bike(String vehicleId, String driverName, double ratePerKm) {
        super(vehicleId, driverName, ratePerKm);
    }

    //implementation of calculateFare() method
    @Override
    public double calculateFare(double distance) {
        return getRatePerKm() * distance; // No extra base charge for bikes
    }

    //implementation of getCurrentLocation() method
    @Override
    public String getCurrentLocation() {
        return currentLocation;
    }

    //implementation of updateLocation() method
    @Override
    public void updateLocation(String location) {
        this.currentLocation = location;
    }
}
```

```
package com.ridehailingapplication;

class Auto extends Vehicle implements GPS {
    private String currentLocation;

    // Constructor to initialize values
    public Auto(String vehicleId, String driverName, double ratePerKm) {
        super(vehicleId, driverName, ratePerKm);
    }

    //implementation of calculateFare() method
    @Override
    public double calculateFare(double distance) {
        return getRatePerKm() * distance + 5; // Adding a minimal base charge for
autos
    }

    //implementation of getCurrentLocation() method
```

```
    @Override
    public String getCurrentLocation() {
        return currentLocation;
    }

    //implementation of updateLocaation() method
    @Override
    public void updateLocation(String location) {
        this.currentLocation = location;
    }
}
```

```
package com.ridehailingapplication;

public class Controller {
    public static void main(String[] args) {
        // Creating Vehicles
        Vehicle car = new Car("CAR123", "Raj", 15);
        Vehicle bike = new Bike("BIKE456", "Veer", 8);
        Vehicle auto = new Auto("AUTO789", "Mayank", 10);

        // Updating Locations
        ((GPS) car).updateLocation("Bhopal");
        ((GPS) bike).updateLocation("Sehore");
        ((GPS) auto).updateLocation("Govindpura");

        // Processing Rides
        Vehicle[] vehicles = {car, bike, auto};
        double distance = 12.5; //distance for ride calculation

        for (Vehicle vehicle : vehicles) {
            System.out.println(vehicle.getVehicleDetails());
            System.out.println("Current Location: " + ((GPS)
vehicle).getCurrentLocation());
            System.out.println("Fare for " + distance + " km: $" +
vehicle.calculateFare(distance));
            System.out.println("----------------------------------------");
        }
    }
}
```

CodInClub

Powered By BridgeLabz